Accenture Data Engineer Interview

(0–5 Years Experience)

Summary

Category	No. of Questions
SQL	1
PySpark	3
Delta / ADF / Cloud	3
Optimization	2
Behavioral	1

1. What they asked me

You are given two tables — `Customers` and `Orders`.
Write a SQL query to find customers who placed **more than 2 orders** in the last month.

What I said

```
SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= DATEADD(MONTH, -1, GETDATE())
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(o.order_id) > 2;
```

Tips

Accenture often checks for **GROUP BY + HAVING** logic.
Mention that you can parameterize the date filter for automation.

---

2. What they asked me

How do you handle **slowly changing dimensions (SCD Type 2)** in Delta Lake?

What I said

I used the **MERGE** command in Delta:

```
deltaTable.alias("tgt").merge(
    updatesDf.alias("src"),
    "tgt.id = src.id"
).whenMatchedUpdate(set = {
    "end_date": "current_date()"
}).whenNotMatchedInsert(values = {
    "id": "src.id",
    "name": "src.name",
    "start_date": "current_date()",
    "end_date": "9999-12-31"
}).execute()
```

Tips

They test for real implementation.
Always link SCD Type 2 to **auditability** and **historical data tracking**.

---

3. What they asked me

What is the difference between **partitioning** and **bucketing** in Spark, and when would you use each?

What I said

- **Partitioning:** divides data into folders based on column values (improves read performance).
- **Bucketing:** divides data into fixed-size buckets using hash (optimizes joins).

Example:

```
df.write.partitionBy("country").parquet("/data/partitioned/")
df.write.bucketBy(8, "customer_id").saveAsTable("bucketed_table")
```

Tips

They check your Spark optimization depth.
Mention you use **partitioning** for filtering, **bucketing** for joining.

---

4. What they asked me

Design a data pipeline that ingests files from **Azure Blob Storage**, transforms them using **PySpark**, and loads into **Snowflake**.

What I said

I walked them through:

1. **Extract:** ADF copy activity from Blob → ADLS raw zone
2. **Transform:** PySpark job in Databricks cleans, deduplicates, standardizes data
3. **Load:** Write to Snowflake via `spark-snowflake` connector
4. **Monitor:** Log success/failure + trigger email alerts

Tips

They want to hear **step-by-step logical flow** — not just tool names.
Mention **metadata-driven pipelines** for bonus points.

---

5. What they asked me

What is **data skew**, and how do you fix it in Spark?

What I said

Data skew occurs when a few partitions have most of the data, causing slow tasks.

Fixes:

- Use **salting technique** (add random key)
- Apply **repartition()**
- Use **broadcast joins** if one dataset is small
- Tune **spark.sql.shuffle.partitions**

Tips

They love when you mention **Spark UI** for identifying skew.
Show real debugging thinking, not theory.

---

6. What they asked me

Explain **Schema Enforcement vs Schema Evolution** in Delta Lake.

What I said

- **Schema Enforcement:** Prevents writing data that doesn't match schema.
- **Schema Evolution:** Allows new columns to be added automatically.

```
df.write.option("mergeSchema",
"true").format("delta").mode("append").save(path)
```

Tips

Accenture tests understanding of **data governance** and **backward compatibility** here.

---

7. What they asked me

Write a PySpark code to find **top 3 products by sales** in each region.

What I said

```
from pyspark.sql.window import Window
from pyspark.sql.functions import col, rank

windowSpec = Window.partitionBy("region").orderBy(col("sales").desc())

df_top3 = df.withColumn("rank", rank().over(windowSpec)).filter(col("rank")
<= 3)
```

Tips

They want to see **window function knowledge** and how you think in groups.

---

8. What they asked me

What happens when your ADF pipeline runs successfully, but data volume in target is less?

What I said

I said I'd:

1. Compare **record counts** between source and target.
2. Verify filter conditions or incremental logic.
3. Check **Lookup Activity / parameters** used.
4. Validate **sink connection or key mapping**.

Tips

This tests your **real production troubleshooting mindset**.

---

9. What they asked me

What is the **difference between ETL and ELT** and which approach is better for cloud?

What I said

Process	Transformation Stage	Example
ETL	Before loading to target	Legacy systems (Informatica)
ELT	After loading to warehouse	Modern tools (ADF + Snowflake)

I said **ELT** is better for scalability in cloud since compute is handled by target systems.

Tips

Always mention **ADF + Databricks + Snowflake** together when they talk about ELT — Accenture uses this stack extensively.

---

10. What they asked me

Tell me about a real project challenge and how you solved it.

What I said

I mentioned a case where incremental loads were failing due to **duplicate records** from API source.
I:

- Created a **deduplication layer** using PySpark (`dropDuplicates`)
- Added a **unique constraint logic** in the staging table
- Implemented **idempotent loads** (safe re-runs without duplication)

Tips

Behavioral + technical blend — show ownership, not just fixes.
They assess **problem-solving maturity** here.